IES RIBERA DE CASTILLA

PRÁCTICA FINAL DE CURSO

Sistema RAG Local con LM Studio y LangChain

Agentes de Inteligencia Artificial – Nivel Fundamentos

Asignatura:	Sistemas Aprendizaje Automático + Modelos Inteligencia Artificial
Herramientas:	LM Studio + Python 3.10+ + LangChain + ChromaDB
Modelo LLM:	Mistral-7B-Instruct-v0.3 (recomendado) – ejecución 100% local
Duración:	3 semanas
Modalidad:	Individual
Calificación:	10 puntos

1. Descripción General del Proyecto

El objetivo de esta práctica es implementar un sistema RAG (Retrieval-Augmented Generation) completamente local: sin APIs de pago, sin enviar datos a la nube y con un LLM que se ejecuta en el propio equipo. El alumnado construirá un chatbot capaz de responder preguntas sobre un conjunto de documentos propios, aplicando los conceptos de agentes y tools vistos en clase.

LM Studio actúa como servidor de inferencia local compatible con la API de OpenAI. LangChain orquesta la cadena RAG, los embeddings y el agente. ChromaDB almacena los vectores de los documentos en disco.

2. Contexto del Problema

Una empresa de formación quiere que sus empleados puedan consultar el manual interno de procedimientos sin depender de un buscador genérico. El sistema debe:

- Leer e indexar documentos en PDF o TXT proporcionados por la empresa.
- Responder preguntas en lenguaje natural basándose exclusivamente en esos documentos.
- Indicar explícitamente cuando la respuesta no está en los documentos (sin alucinar).
- Ofrecer al menos una funcionalidad extra mediante una Tool de agente (p. ej. consultar la fecha actual, resumir un documento o buscar una palabra clave).

El alumnado simula este escenario usando cualquier conjunto de documentos de su elección (mínimo 3 ficheros PDF o TXT, mínimo 10 páginas en total).

3. Arquitectura del Sistema

El sistema tiene cinco capas que el alumnado debe implementar:

#	Capa	Descripción y tecnología
1	Ingesta de documentos	DocumentLoader (PyPDFLoader / TextLoader) + RecursiveCharacterTextSplitter → chunks de texto
2	Embeddings y vectorstore	HuggingFaceEmbeddings (modelo local) → ChromaDB persistido en disco
3	Retriever	ChromaDB.as_retriever(search_kwargs={'k': 4}) → recupera los chunks más relevantes
4	Cadena RAG	ChatOpenAI apuntando a LM Studio (base_url local) + PromptTemplate + RetrievalQA o LCEL chain
5	Agente + Tool	create_react_agent / AgentExecutor + al menos una Tool personalizada con @tool

Flujo de una consulta

Usuario escribe pregunta → Agente decide si usar la Tool RAG u otra tool
Tool RAG: embeddings de la pregunta → búsqueda vectorial → k chunks recuperados
LLM local (LM Studio) recibe: prompt de sistema + chunks + pregunta → genera respuesta
Respuesta devuelta al usuario con las fuentes citadas

4. Software y Modelo Recomendado

4.1 LM Studio

LM Studio (https://lmstudio.ai) permite descargar y ejecutar modelos GGUF localmente y expone un servidor compatible con la API de OpenAI en http://localhost:1234/v1.

Pasos de configuración:

1. Descarga e instala LM Studio para tu sistema operativo.
2. En la pestaña 'Discover', busca y descarga el modelo recomendado (ver tabla abajo).
3. Ve a la pestaña 'Local Server' y pulsa 'Start Server'.
4. Verifica que el servidor responde: abre http://localhost:1234/v1/models en el navegador.

4.2 Modelos recomendados

Modelo	Tamaño en disco	RAM mínima	Por qué es adecuado
Mistral-7B-Instruct-v0.3 (Q4_K_M)	~4,1 GB	8 GB	Equilibrio ideal velocidad/calidad. Sigue instrucciones con precisión. Recomendado para la práctica.
LLaMA-3.1-8B-Instruct (Q4_K_M)	~4,7 GB	8 GB	Excelente comprensión de contexto largo. Buena alternativa si hay más RAM disponible.
Phi-3-mini-4k-instruct (Q4)	~2,2 GB	4 GB	Muy ligero. Válido para equipos con poca RAM, aunque la calidad es algo inferior.

⚠ **Atención – requisitos mínimos de hardware**

RAM: mínimo 8 GB (16 GB recomendado para modelos de 7-8B).
Disco: mínimo 6 GB libres para el modelo + vectorstore.
GPU: no es obligatoria; LM Studio funciona en CPU, aunque más lento.
Si el equipo tiene menos de 8 GB de RAM, usa Phi-3-mini (ver tabla).

4.3 Entorno Python

Crea un entorno virtual e instala las dependencias:

```
# Crear entorno virtual
python -m venv venv
source venv/bin/activate        # Linux/Mac
venv\Scripts\activate           # Windows

# Instalar dependencias
pip install langchain langchain-community langchain-openai
pip install chromadb sentence-transformers
pip install pypdf tiktoken
```

5. Requisitos del Sistema – Apartados

El proyecto se divide en cinco apartados técnicos que corresponden directamente a los criterios de la rúbrica. Cada apartado incluye código de referencia orientativo que el alumnado debe adaptar y completar.

Apartado A – Carga e indexación de documentos

Crea el script ingest.py que carga los documentos, los divide en chunks y construye el vectorstore.

```
# ingest.py
from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
import os

DOCS_PATH = "./docs"          # Carpeta con tus PDFs y TXTs
DB_PATH   = "./chroma_db"     # Donde se guardará el índice

def cargar_documentos(ruta):
    documentos = []
    for fichero in os.listdir(ruta):
        ruta_completa = os.path.join(ruta, fichero)
        if fichero.endswith(".pdf"):
            loader = PyPDFLoader(ruta_completa)
        elif fichero.endswith(".txt"):
            loader = TextLoader(ruta_completa, encoding="utf-8")
        else:
            continue
        documentos.extend(loader.load())
    return documentos

def indexar(documentos):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,      # TODO: justifica este valor en el informe
```

```
        chunk_overlap=50     # TODO: justifica este valor en el informe
    )
    chunks = splitter.split_documents(documentos)
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
    db = Chroma.from_documents(chunks, embeddings, persist_directory=DB_PATH)
    print(f"Indexados {len(chunks)} chunks en {DB_PATH}")
    return db

if __name__ == "__main__":
    docs = cargar_documentos(DOCS_PATH)
    indexar(docs)
```

Puntos clave del Apartado A

chunk_size=500 significa que cada trozo tiene máximo 500 caracteres. Prueba con 300 y 800 y compara la calidad de las respuestas.
chunk_overlap=50 evita que se pierda contexto entre chunks consecutivos.
all-MiniLM-L6-v2 es un modelo de embeddings ligero que se descarga automáticamente la primera vez.
El vectorstore se guarda en disco: no hay que re-indexar cada vez que se ejecuta el sistema.

Apartado B – Cadena RAG

Crea el script rag_chain.py con la cadena que conecta el retriever con el LLM local.

```
# rag_chain.py
from langchain_openai import ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate

DB_PATH = "./chroma_db"
LM_STUDIO_URL = "http://localhost:1234/v1"

PROMPT_TEMPLATE = """
Eres un asistente experto. Usa ÚNICAMENTE el siguiente contexto para responder.
Si la respuesta no está en el contexto, di: 'No tengo información sobre eso.'

Contexto:
{context}

Pregunta: {question}
Respuesta:
"""

def crear_cadena_rag():
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
    db = Chroma(persist_directory=DB_PATH, embedding_function=embeddings)
    retriever = db.as_retriever(search_kwargs={"k": 4})

    llm = ChatOpenAI(
        base_url=LM_STUDIO_URL,
        api_key="lm-studio",   # LM Studio no valida la clave
```

```
        model="local-model",
        temperature=0.2        # TODO: prueba 0.0 y 0.7, observa diferencias
    )

    prompt = PromptTemplate(
        input_variables=["context", "question"],
        template=PROMPT_TEMPLATE
    )

    cadena = RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever,
        return_source_documents=True,
        chain_type_kwargs={"prompt": prompt}
    )
    return cadena
```

Parámetro temperature

temperature=0.0 → respuestas deterministas y conservadoras (ideal para RAG).
temperature=0.7 → respuestas más creativas pero con más riesgo de alucinación.
Para un sistema RAG documental se recomienda temperature entre 0.0 y 0.3.

Apartado C – Tool de agente

Crea el script agent.py que define al menos una Tool personalizada y un agente que decide cuándo usarla.

```
# agent.py
from langchain.tools import tool
from langchain.agents import AgentExecutor, create_react_agent
from langchain import hub
from langchain_openai import ChatOpenAI
from rag_chain import crear_cadena_rag
from datetime import datetime

LM_STUDIO_URL = "http://localhost:1234/v1"

# -- Tool 1: RAG sobre los documentos --
@tool
def consultar_documentos(pregunta: str) -> str:
    """Consulta la base de conocimiento interna para responder preguntas
    sobre los documentos indexados."""
    cadena = crear_cadena_rag()
    resultado = cadena.invoke({"query": pregunta})
    return resultado["result"]

# -- Tool 2 (ejemplo): fecha y hora actual --
@tool
def obtener_fecha_actual(dummy: str = "") -> str:
    """Devuelve la fecha y hora actual del sistema."""
    return datetime.now().strftime("%A, %d de %B de %Y - %H:%M")

# TODO: crea al menos UNA tool adicional propia

def crear_agente():
    llm = ChatOpenAI(
        base_url=LM_STUDIO_URL,
```

```
        api_key="lm-studio",
        model="local-model",
        temperature=0.1
    )
    tools = [consultar_documentos, obtener_fecha_actual]
    prompt = hub.pull("hwchase17/react")
    agente = create_react_agent(llm, tools, prompt)
    return AgentExecutor(agent=agente, tools=tools, verbose=True)

if __name__ == "__main__":
    executor = crear_agente()
    while True:
        pregunta = input("\nTú: ")
        if pregunta.lower() in ["salir", "exit"]: break
        respuesta = executor.invoke({"input": pregunta})
        print(f"\nAsistente: {respuesta['output']}")
```

⚠ Errores frecuentes en el Apartado C

Si el agente entra en bucle infinito, añade max_iterations=5 al AgentExecutor.
Si LangChain no encuentra el prompt de hub, instala: pip install langchainhub
La docstring de cada @tool es CRÍTICA: el agente la lee para saber cuándo usar la tool.
Si el LLM local no soporta bien ReAct, prueba con create_openai_tools_agent en su lugar.

Apartado D – Estructura del proyecto

El proyecto debe entregarse con la siguiente estructura de carpetas:

```
rag_project/
├── docs/                    # Tus documentos PDF y TXT
│   ├── documento1.pdf
│   └── documento2.txt
├── chroma_db/               # Generado por ingest.py (incluir en el ZIP)
├── ingest.py                # Apartado A: carga e indexación
├── rag_chain.py             # Apartado B: cadena RAG
├── agent.py                 # Apartado C: agente + tools
├── requirements.txt         # pip freeze > requirements.txt
└── README.md                # Instrucciones de instalación y uso
```

El README.md debe incluir como mínimo:

- Instrucciones para instalar dependencias (pip install -r requirements.txt).
- Cómo configurar LM Studio (modelo a usar, URL del servidor).
- Orden de ejecución: primero ingest.py, luego agent.py.
- Dos o tres ejemplos de preguntas que el sistema responde correctamente.

6. Informe Escrito

El informe documenta las decisiones técnicas tomadas. Extensión máxima: 8 páginas PDF (sin portada).

Estructura obligatoria:

5. Portada: nombre(s), fecha, asignatura, título del proyecto.
6. Descripción del sistema: qué hace el RAG y para qué conjunto de documentos.
7. Arquitectura implementada: diagrama o descripción de las 5 capas.

8. Justificación del modelo: por qué elegiste ese LLM y esos parámetros (chunk_size, k, temperature).
9. Resultados: 3-5 ejemplos de preguntas con las respuestas obtenidas y evaluación de su calidad.
10. Limitaciones y mejoras: qué no funciona bien y qué harías con más tiempo.

Consejo para el informe

La sección de justificación es la más valorada: explica POR QUÉ elegiste cada parámetro, no solo cuál usaste.
Incluye al menos un ejemplo de pregunta donde el sistema responde 'No tengo información' correctamente.

7. Entregables y Plazos

Entregable	Contenido mínimo	Formato / Canal
Código fuente	Carpeta con todos los scripts .py, requirements.txt y README de instalación	ZIP en aula virtual
Vectorstore	Carpeta chroma_db/ (o equivalente) con el índice generado	Incluido en el ZIP
Informe escrito	Arquitectura, modelo elegido, parámetros, resultados, limitaciones (máx. 8 páginas)	PDF en aula virtual
Demostración en vivo	El sistema responde 3 preguntas sobre el documento en clase	Presencial / videoconferencia

Aporte extra: implementa memoria conversacional con ConversationBufferMemory o RunnableWithMessageHistory para que el agente recuerde el historial de la sesión.

8. Planificación Semanal Sugerida

Semana	Fase	Tareas principales	Entregable parcial
1	**Configuración y carga de datos**	Instalar LM Studio + modelo. Configurar entorno Python. Crear pipeline de carga, chunking e indexación. Probar embeddings.	Vectorstore persistido en disco + script de indexación funcional
2	**Cadena RAG + Tool**	Implementar retriever y cadena RAG con prompt personalizado. Crear Tool personalizada e integrarla en un agente LangChain.	Script rag_chain.py y agent.py ejecutables con respuestas coherentes
3	**Refinamiento, informe y defensa**	Ajustar parámetros (k, chunk_size, temperatura). Refactorizar código. Redactar informe. Preparar demostración en vivo.	Entrega final: código + informe PDF + demostración

9. Criterios de Evaluación – Rúbrica Detallada

Cada fila describe exactamente lo que se espera para obtener la puntuación máxima en ese criterio. Léela antes de empezar.

Criterio de Evaluación	Indicadores de Logro	Pts máx.	Peso	Escala de puntuación
1. Entorno y configuración	– LM Studio instalado con modelo descargado y funcionando – Servidor local OpenAI-compatible activo en puerto 1234 – Entorno Python con dependencias instaladas sin errores	**1,0**	10%	*0 pts: no funciona* *0,5 pts: parcialmente configurado* *1,0 pts: todo operativo*
2. Carga e indexación de documentos	– Documentos cargados con DocumentLoader adecuado – Texto dividido correctamente con TextSplitter (chunk_size y overlap razonados) – Vectorstore creado con embeddings y persistido en disco	**2,0**	20%	*0 pts: sin indexación* *1,0 pt: carga sin chunking correcto* *2,0 pts: pipeline completo y razonado*
3. Cadena RAG funcional	– Retriever conectado al vectorstore – Prompt personalizado con contexto e historial – Chain devuelve respuestas coherentes con el documento	**2,5**	25%	*0 pts: no responde* *1,0 pt: responde sin contexto* *2,5 pts: respuestas fundamentadas en el documento*
4. Tool de agente integrada	– Al menos una Tool personalizada creada con @tool – Agente inicializado con la tool y el LLM local – El agente decide cuándo usar la tool correctamente	**2,0**	20%	*0 pts: sin tool* *1,0 pt: tool creada pero no integrada* *2,0 pts: agente + tool funcionales*
5. Calidad del código	– Código estructurado en módulos o funciones claras – Variables y funciones con nombres descriptivos – Comentarios que explican decisiones clave	**1,0**	10%	*0 pts: ilegible* *0,5 pts: parcialmente organizado* *1,0 pt: limpio, modular y comentado*
6. Memoria / Informe escrito	– Describe arquitectura RAG implementada – Justifica elección del modelo y parámetros – Reflexiona sobre limitaciones y mejoras posibles	**1,0**	10%	*0 pts: no entregado* *0,5 pts: incompleto* *1,0 pt: completo y reflexivo*
7. Demostración y defensa	– Demuestra el sistema respondiendo preguntas en vivo – Explica por qué el LLM responde lo que responde – Responde preguntas del profesor con fundamento	**0,5**	5%	*0 pts: no defiende* *0,25 pts: demostración sin explicación* *0,5 pts: dominio claro*

Criterio de Evaluación	Indicadores de Logro	Pts máx.	Peso	Escala de puntuación
	TOTAL	10	100%	

Escala cualitativa

0 – 4,9	5 – 5,9	6 – 6,9	7 – 8,9	9 – 10
Insuficiente	Suficiente	Bien	Notable	Sobresaliente

10. Recomendaciones Finales

Buenas prácticas

- Ejecuta ingest.py solo cuando añadas o cambies documentos; el vectorstore se reutiliza.
- Usa un .env o variables de entorno para la URL de LM Studio, no la hardcodees.
- Prueba primero la cadena RAG sola (rag_chain.py) antes de integrarla en el agente.
- Si el modelo local es lento, reduce k de 4 a 2 y chunk_size a 300.
- Haz commits frecuentes a Git para no perder trabajo.